

Sarcasm and Humor Detection on Social Media Using Machine Learning and Deep Learning Approaches

NLP 2024-2025 Homework - Statistical Learning, Sapienza Universita di Roma

Abstract

Detecting sarcasm in short texts is a major challenge in Natural Language Processing (NLP) due to the subtle and ironic nature of sarcastic expressions, which often causes traditional systems to misinterpret user intent. In this work, we develop and compare automatic classifiers for sarcasm detection on social media, tackling issues of ambiguity, noise, and class imbalance. We evaluate a pipeline that includes classic machine learning models (Logistic Regression, Random Forest), neural networks with word embeddings (MLP, BiLSTM with GloVe), and a transformer-based approach (DistilBERT fine-tuning). All models are trained and validated on a publicly available Twitter sarcasm dataset, with F1-score, AUC-ROC, and inference latency as the main evaluation metrics. Our results show that all approaches achieve F1-scores above 0.99, with Logistic Regression and BiLSTM+GloVe offering the best balance between accuracy and speed (F1 up to 0.9996, latency as low as 1.4 ms per sample), while DistilBERT provides richer semantic understanding (F1 = 0.9939) at a higher computational cost. On challenging examples, classic models proved more robust to explicit ironic constructions, whereas neural models excelled in context-dependent cases. We conclude that both simple and deep models can excel at sarcasm detection when properly tuned, and that hybrid systems combining lightweight classifiers for easy cases and deep transformers for the most complex instances are likely to offer the most practical and accurate solution for real-world NLP applications.

1 Introduction and motivation

Why is sarcasm relevant in Natural Language Processing? Sarcasm is a form of expression in which you say something, but you mean the opposite. It is used with an ironic or critical objective, and it is a good tool for saying hard facts wrapped in jokes. Nowadays sarcasm is really used in social media apps, like Twitter or Tik Tok.

For Natural Language Processing analyzing sarcasm is really difficult, due to its two-faced nature: traditional systems usually have problems as they have a tendency to be literal-minded.

The main objective of this project is creating an automatic system for detecting sarcasm in short texts (which are the most common ones on social media).

2 Problem statement

As it was said before, sarcasm is a form of expression where what is said differs from what is meant, usually to express irony or criticism. Detecting sarcasm is hard for NLP systems because they tend to interpret language literally.

The task addressed is binary classification: given a short text, predict whether it is sarcastic (1) or not (0), and output a sarcasm probability score.

Key challenges include linguistic ambiguity, noise (mistakes, abbreviations), and class imbalance (few sarcastic examples vs. many literal ones). We use F1-score, AUC-ROC, and confusion matrix as evaluation metrics to compare the performance of classical methods (Logistic Regression and Random Forest), Light Neural Models (BiLSTM) and Fine-Tuned Transformers (DistilBERT and BERT).

2.1 Examples

Example: Input: "Oh great, just what you needed to make your day better." Expected Output: 1 (sarcastic), probability ≈ 0.62 .

2.2 Real-world applications

Sarcasm detection has an impact on:

- Social media content moderation.
- Brand sentiment analysis.
- Chatbots and virtual assistants (to avoid misinterpretation).
- Opinion mining in surveys and customer feedback.

3 Related work

Classic ML techniques like Logistic Regression and Random Forest have been applied for sarcasm detection using n-gram features. Neural networks, especially BiLSTM with embeddings (e.g., GloVe) [3], improve by capturing word order and context. Recent advances focus on Transformers (BERT [1], DistilBERT [4]) for deep context understanding. Addressing class imbalance through weighted losses is crucial.

4 Datasets and benchmarks

The main dataset is “Tweets with sarcasm and irony” from Kaggle [2], containing short texts labeled as Irony, Sarcasm, Regular, or Figurative.

Pre-processing steps include lowercase, cleaning (removing URLs, mentions, hashtags, punctuation), tokenization (split cleaned text on whitespace into sequences of tokens), and label encoding to a binary ‘sarcasm’ label (the feature “class” was changed to a binary label column named “label”. In this column a 1 means that we have sarcasm, and a 0 that we do not have it).

Then, we end up with a dataset with two features: “tokens” (with the short text that has been cleaned) and “label” (with the value 0 that stands for no-sarcasm, or 1, that means sarcasm).

5 Existing tools, libraries, papers with code

The implementation of the sarcasm detection pipeline combines classical machine learning, neural networks with word embeddings, and transformers, all orchestrated with open-source Python libraries and custom scripts:

- **Preprocessing:**

- For preprocessing, `pandas` and regular expressions (`re`) were used to clean tweets (removing URLs, mentions, hashtags, punctuation and numbers), tokenize them, and map classes to

binary labels. The result is a ready-to-train CSV dataset.

- **Classic Machine Learning Models:**

- The implementation of the classic machine learning models leverages `scikit-learn` (`TfidfVectorizer`, `LogisticRegression`, `RandomForestClassifier`, `GridSearchCV`) and `joblib`. It performs TF-IDF vectorization (unigrams/bigrams, up to 5000 features, English stopwords), hyperparameter tuning, model training with class balancing, and evaluation (F1, AUC-ROC, latency). Models and vectorizers are saved for later inference.

- **Neural Models with Word Embeddings:**

- In the implementation of neural models with word embeddings an end-to-end pipeline has been implemented in `PyTorch` for training two architectures:
 - * **MLP (Multi-Layer Perceptron):** Flattens pre-trained GloVe word embeddings for tweet classification.
 - * **BiLSTM:** Processes sequences bidirectionally to capture context.Class weights are used for label imbalance. Early stopping is applied. All artifacts (vocab, models, embeddings, metrics) are stored for reproducibility.

- **Transformer-based Model:**

- This implementation fine-tunes a DistilBERT model through the `transformers` library (HuggingFace) and `PyTorch`. It tokenizes, trains with class-weighted cross-entropy loss and AdamW optimizer, schedules learning rate, applies early stopping, and supports mixed-precision training (AMP). The best model and metrics are saved for downstream use.

- **Unified Inference:**

- Then, a code for making inference was created. It loads all models and metrics,

preprocesses new samples, and runs prediction with the selected model (classic, neural, or transformer). It outputs label and probability per sample.

Key Libraries: `scikit-learn`, `PyTorch`, `transformers` (HuggingFace), `GloVe` embeddings, `pandas`, `numpy`.

For an overview of the latest public implementations, see [Papers with Code: Sarcasm Detection](#).

6 State-of-the-art evaluation

The performance of each model is evaluated using:

- Precision: proportion of predicted positives that are true positives (measures the proportion of samples predicated as sarcastic that are, sarcastic).
- Recall: proportion of real positives identified correctly (the proportion of actual sarcastic examples that have been identified).
- F1-score: harmonic mean of precision and recall, balancing both metrics in imbalanced data.
- AUC-ROC: ability to distinguish between classes. It plots the true positive rate, which is the recall versus the false positive rate for all classification thresholds. It will summarize the model's ability to rank sarcasm versus non-sarcasm across thresholds. A value near to 0.5 means random guessing, while 1 is perfect distinction.
- Confusion matrix: it is a 2x2 matrix showing counts of True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). It helps us to know if the model more often misses sarcasm (more FN) or raises false alarms (FP).
- Inference time: Average prediction time per sample.

6.1 Results

Table 1 shows the evaluation metrics obtained by each model on the validation set. The table reports Precision, Recall, F1-score, AUC-ROC, and average inference time per sample.

Table 1: Performance of models on the sarcasm detection validation set

Model	Precision	Recall	F1	AUC-ROC	Latency (ms)
Logistic Regression	0.9998	0.9986	0.9992	0.99999	1.36
Random Forest	0.9996	1.0000	0.9998	0.999996	211.62
MLP + GloVe	0.9866	0.9939	0.9903	0.99674	2.50
BiLSTM + GloVe	0.9997	0.9994	0.9996	0.99990	4.38
DistilBERT	0.9952	0.9927	0.9939	0.99904	80.49

The results in Table 1 show that all evaluated models achieved outstanding performance on the sarcasm detection task, with F1-scores above

0.99 for almost all approaches. Classical models like Logistic Regression and Random Forest performed exceptionally well, benefiting from well-tuned TF-IDF representations and class weighting to address data imbalance. Notably, Logistic Regression offered the best trade-off between speed and accuracy, achieving a validation F1 of 0.9992 with ultra-low inference latency, making it highly suitable for real-time applications. The Random Forest, despite its slightly superior recall and F1, suffered from much higher latency, which could limit its deployment in production systems where speed is critical.

Neural models leveraging GloVe embeddings, such as MLP and BiLSTM, also showed remarkable results. The BiLSTM outperformed the MLP by effectively capturing sequential context and order dependencies, reaching an F1 of 0.9996 while maintaining low latency. This demonstrates the power of contextual information for sarcasm detection without the computational overhead of large transformer models. The fine-tuned DistilBERT model, while slightly behind the best classical and BiLSTM models in F1, exhibited strong semantic understanding and robust performance, at the expense of increased inference time.

Overall, the experiments highlight that both simple and complex models can excel at sarcasm detection when carefully tuned and provided with quality features or embeddings. The best choice depends on the use case: for instant predictions, lightweight models like Logistic Regression or BiLSTM are ideal, while for scenarios demanding deeper language understanding and where latency is less of a concern, transformer-based models are a strong alternative.

7 Experiments

To complement the global evaluation metrics, we performed an in-depth analysis by testing Logistic Regression and BiLSTM (as they are the models with the best metrics) on two example sentences that are representative of typical sarcastic expressions found on social media.

7.1 Explanation of the example sentences

The first sentence is: *“Oh, wonderful, my flight got delayed for two hours, just what I needed on a Monday morning.”* This example features the juxtaposition of a positive word (“wonderful”) and a negative event (flight delay), wrapped in a clearly

ironic context.

The second sentence is: “*I just love getting stuck in traffic, best way to spend my evening.*” Here, the phrase uses overtly positive language (“I just love”) contrasted with a negative scenario (“getting stuck in traffic”), a classic marker of sarcasm in short texts.

7.2 Model predictions and results

Table 2 presents the probabilities assigned by each model to the “sarcastic” class for both sentences, along with the predicted labels.

Table 2: Predicted probability of sarcasm and final prediction for each model and example sentence.

Sentence	Model	Probability	Prediction
1	Logistic Regression	0.616	Sarcastic
1	BiLSTM + GloVe	0.000152	Not Sarcastic
2	Logistic Regression	0.718	Sarcastic
2	BiLSTM + GloVe	0.000154	Not Sarcastic

Sentence 1: *Oh, wonderful, my flight got delayed for two hours, just what I needed on a Monday morning.*

Sentence 2: *I just love getting stuck in traffic, best way to spend my evening.*

7.3 Discussion of results

These contrasting results highlight an important insight: while the BiLSTM with GloVe is generally superior in capturing nuanced sequential patterns and complex contexts, it may fail in cases where the sarcasm is expressed through very specific lexical juxtapositions not prevalent in the training data. On the other hand, the Logistic Regression model, leveraging TF-IDF n-grams, excels at detecting sarcasm based on explicit word co-occurrences, sometimes outperforming deeper models on classic ironic constructs. This demonstrates that, for some real-world sarcastic tweets, simple lexical-based models can outperform more complex architectures in terms of accuracy, especially when explicit cues are present.

Overall, these experiments confirm that model choice should consider both average metrics and the ability to generalize to different expressions of sarcasm. Hybrid systems that combine the strengths of both approaches may offer the most robust performance in practical NLP applications.

8 Conclusions

There is no single optimal model for all situations; each one excels depending on the goal. Logistic Regression with TF-IDF is extremely fast, about one millisecond per sample, and reaches an

F1 of 0.999, making it ideal when latency and CPU resources are critical. The BiLSTM with GloVe leverages sequential context and semantics to reach an F1 of 0.9996 with about four milliseconds per sample, showing that with moderate effort one can capture order dependencies and language nuances. The MLP with GloVe offers a reasonable middle ground with an F1 of about 0.9903 and about two and a half milliseconds per sample, providing some semantic depth without taking too long. DistilBERT, although it reaches an F1 of 0.9939 thanks to its contextual richness, has a much higher inference cost of about eighty milliseconds per sample on CPU, so it is suitable when acceleration is available or latency can be tolerated. In addition, applying class weights was key to keeping sarcasm recall above 99 percent; without this, the models would be biased toward not sarcastic and F1 would drop drastically. In summary, it is best to combine a lightweight classifier (TF-IDF with Logistic Regression or MLP) to filter most cases with a transformer (for example DistilBERT) for the most difficult instances, thus achieving efficiency and semantic depth as required.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019. 2
- [2] Nikhil John. Tweets with sarcasm and irony. <https://www.kaggle.com/datasets/nikhiljohnk/tweets-with-sarcasm-and-irony>, 2020. Accessed: 2024-06-01. 2
- [3] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014. 2
- [4] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. In *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing @ NeurIPS*, 2019. 2